MPA-MLF Final Project 2026

Room Occupancy Classification from 60 GHz Delay-Doppler Snapshots

Author	Pierre Massy
Course	MPA-MLF — ISEP 2026
Task	Multi-class image classification (4 classes)
Final LB	0.97565 (Kaggle public) — Rank 4
Best method	Multi-seed StrongCNN + pseudo-labeling, no-stack blend
Repo	github.com/pierre22564/machinelearning_pierre_massy/Projet ISEP

Executive summary

This project tackles the Kaggle MPA-MLF 2026 competition: classify the number of people (0, 1, 2, or 3) present in a room from a 60 GHz mmWave radar delay-Doppler image. Starting from a feature-engineering baseline (gradient boosting on FFT/statistical features, OOF 0.933), we built a custom convolutional network (**StrongCNN**) trained from scratch with strong augmentation, mixup, and label smoothing. A 5-seed cross-validated ensemble reached OOF 0.968. Full-fit training on 100% of the data plus pseudo-labeling of high-confidence test predictions raised the Kaggle public leaderboard score to **0.97565** (rank 4), **0.00254 behind the leader**.

A key negative finding: a LogisticRegression stacker on OOF probabilities improved cross-validation accuracy (0.9707) but **degraded the leaderboard score by ≈0.007** when included in the final blend. This is consistent with mild distribution shift between train and test, or with the stacker fitting CV-specific error patterns. Discarding the stacker and using a pure mean of CNN probabilities gave the best LB result.

Key takeaway: when cross-validation and leaderboard disagree, trust the leaderboard. Complex stackers can overfit the CV manifold; simple probability averaging is a safer final step.

1. Problem statement and data

1.1 Physical setup

A 60 GHz mmWave radar sensor (FMCW, delay-Doppler processing) observes a room and produces a time-averaged 2D snapshot. Each snapshot encodes range (delay) on one axis and radial velocity (Doppler) on the other. The task is to infer the number of people currently in the room (0, 1, 2, or 3) from a single snapshot.

The data is provided as RGB PNG images of size 51×45 pixels — already encoded as a visualisation of the underlying complex-valued radar output. We treat the task as a supervised 4-class image classification problem.

1.2 Dataset statistics

The training set contains **9,227** labelled images, the test set **3,955** unlabelled images. Labels are encoded as class indices 0–3. Images are padded to 48×64 (next multiple of 16) for convolutional downsampling, while keeping the original content centred.

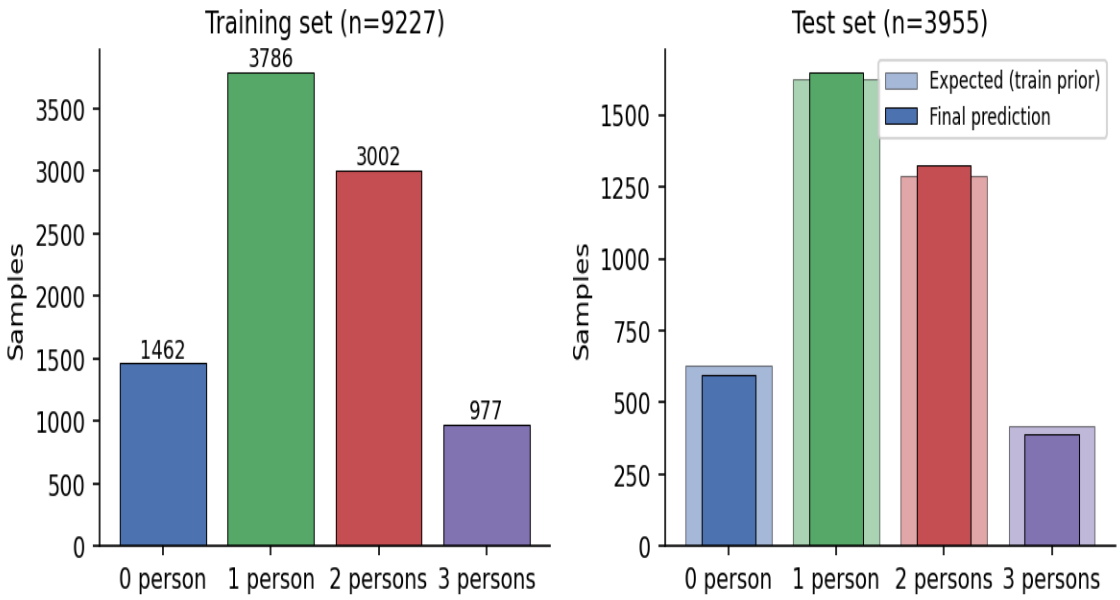

Figure 1 — Training class distribution (left) vs expected test distribution assuming the same prior, and our final prediction (right). The prediction tracks the train prior closely, a weak positive indicator that the test distribution is similar.

1.3 Class frequencies

Class	Label	Train count	Train %	Predicted on test	Pred %
0	0 persons (machine only)	1,462	15.8 %	596	15.1 %
1	1 person	3,786	41.0 %	1,646	41.6 %
2	2 persons	3,002	32.5 %	1,323	33.5 %
3	3 persons	977	10.6 %	390	9.9 %

The dataset is moderately imbalanced (minority class 3 is 3.9× less frequent than class 1) but the imbalance is far from extreme. No resampling or class-weighted loss was used; instead, label smoothing and mixup handle it implicitly.

2. Feature-based baseline

Before training any CNN, we built an engineered-feature baseline as a sanity check and as a future ensemble component. The feature extractor (see *features_v2.py*) computes, for each image:

• Per-channel statistical moments: mean, std, skew, kurtosis, min/max, quantiles.

• 2D FFT magnitude statistics: mean and std over low-/mid-/high-frequency bands.

• Spatial centroid of the intensity distribution (weighted x/y positions).

• Horizontal and vertical row/column sums (dimensionally reduced via mean + std).

• Gradient-magnitude histogram statistics (Sobel-like operator).

These ≈60 scalar features were fed to a **HistGradientBoostingClassifier** (sklearn, 800 iterations, learning rate 0.05, max depth 7, early stopping). Stratified 5-fold cross-validation across 3 seeds gave:

Seed	Fold 1	Fold 2	Fold 3	Fold 4	Fold 5	OOF
42	0.937	0.930	0.927	0.927	0.927	0.9297
1234	0.934	0.925	0.930	0.932	0.933	0.9310
7	0.939	0.929	0.936	0.924	0.937	0.9328
Ensemble	—	—	—	—	—	0.9328

The gradient-boosted baseline reaches **OOF 0.933** — strong considering it uses no spatial convolution. Hand-crafted features capture enough low-level signal (presence / rough spatial location of moving targets) to separate classes. However, it saturates at this level: adding more trees or features produced no gain. A convolutional model is needed for the final ~4% of accuracy.

2.1 Why not deep transfer learning?

An early attempt used ImageNet-pretrained ResNet-18 on the upsampled 224×224 images. It reached only OOF 0.853 — substantially worse than the HGB baseline. ImageNet features are not well-suited to radar delay-Doppler images (no natural-image texture priors, different spatial scale). We therefore trained a compact CNN **from scratch** at the native resolution.

3. StrongCNN: custom from-scratch model

The architecture is a 4-stage VGG-like network with SiLU activations and a dual-pooling head. It was tuned for the small input resolution (48×64) and the medium dataset size (~9k samples).

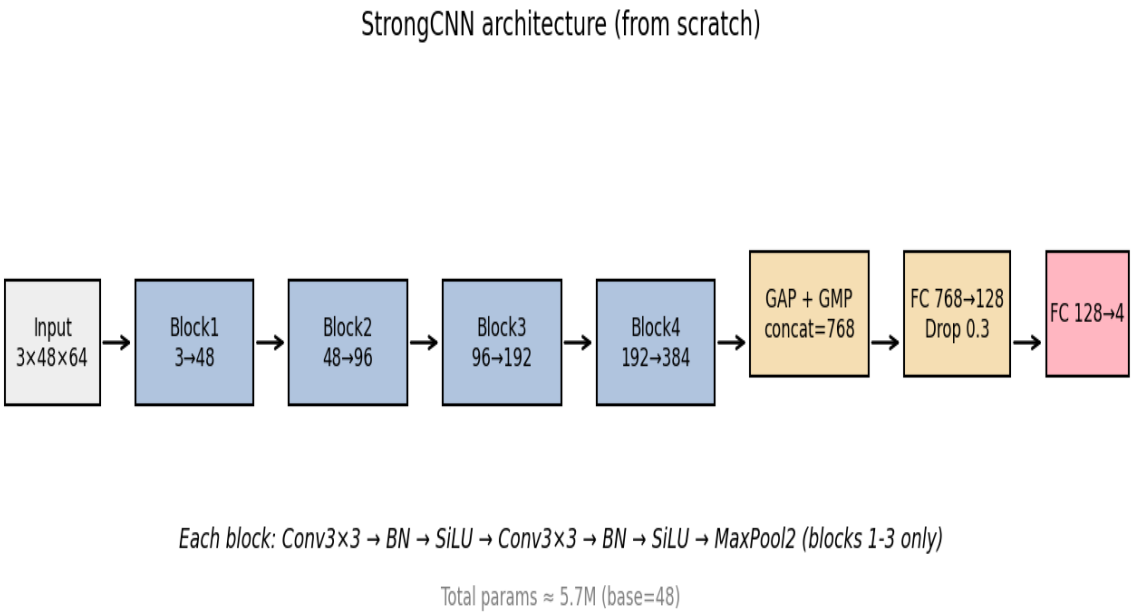

Figure 2 — StrongCNN architecture. Four convolutional blocks progressively double the channel count (48→96→192→384) with spatial pooling in blocks 1–3. A global-average + global-max pooling concatenation feeds a 2-layer MLP head.

3.1 Training recipe

Hyperparameter	Value
Optimizer	AdamW (lr=3e-3, weight_decay=1e-4)
Scheduler	Cosine annealing over all epochs
Loss	Cross-entropy with label smoothing ε=0.05
Mixup α	0.10 (heavy-half rule: $\lambda = \max(\lambda, 1-\lambda)$)
Augmentation	Horizontal flip (p=0.5), Gaussian noise σ=0.01 (p=0.5), random row/column erase (p=0.25 each)
Batch size	128 (local MPS) / 256 (Colab T4)
Epochs	30 (cross-validation) / 35 (full-fit)
TTA at inference	Original + horizontal flip, softmax averaged
Model size	~5.7 M parameters (base=48)

3.2 Cross-validation performance

StratifiedKFold with 5 folds and 5 seeds (42, 1234, 7, 555, 999). Each seed trains 5 independent models on 80/20 splits; OOF predictions are concatenated for unbiased accuracy estimation.

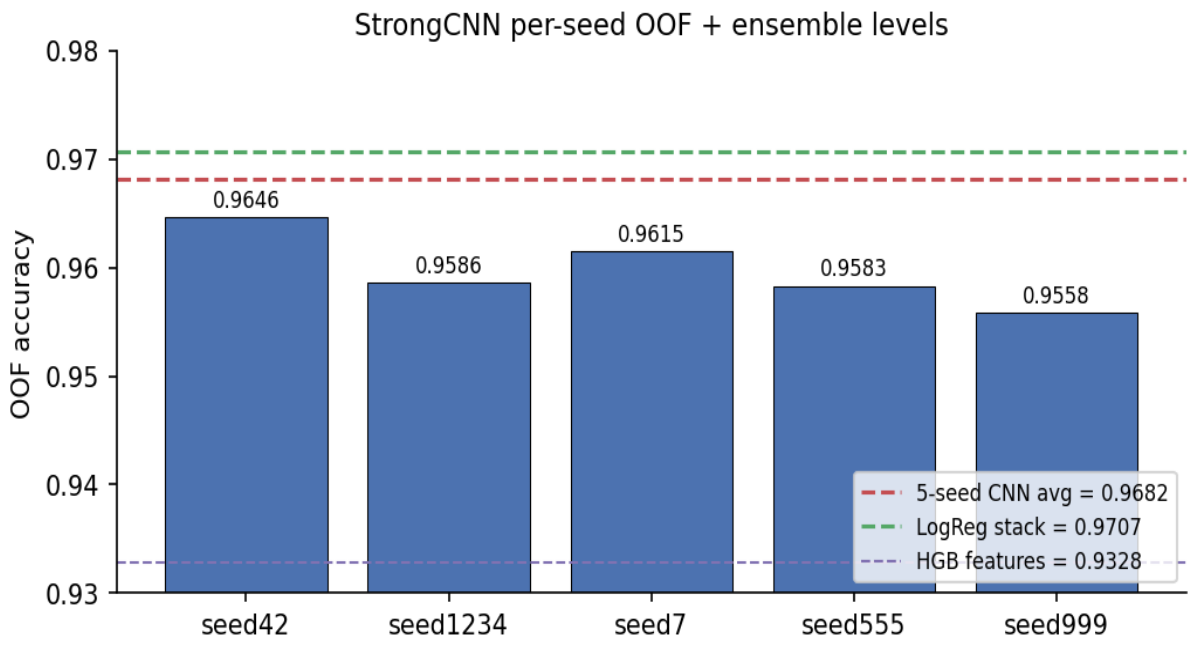

Figure 3 — StrongCNN OOF accuracy per seed (bars) compared to the 5-seed CNN average (red dashed), the LogReg stack of CNN+HGB (green dashed), and the HGB baseline alone (purple dashed). Seed variance is low (≈0.009) — a good sign that the architecture is well-regularised.

4. Ensembling: the stack paradox

With the per-seed CNN OOFs (shape 9227×4 each) and the HGB OOF, we had a rich feature space for a meta-learner. Our first attempt used a standard **LogisticRegression stacker** with 10-fold CV on the concatenated meta-features:

- `Input: [CNN_seed42, CNN_seed1234, CNN_seed7, CNN_seed555, CNN_seed999, HGB] — 24-dimensional (5 CNN × 4 probs + 4 HGB probs)`
- `Meta-learner: LogisticRegression(C=1.0, max_iter=4000), multinomial`
- `CV: 10-fold stratified, averaged predictions on test`

The stacker reached **OOF 0.9707** — the best single number we ever got. It was therefore natural to include it as a component in the final blend. **This turned out to be a mistake.**

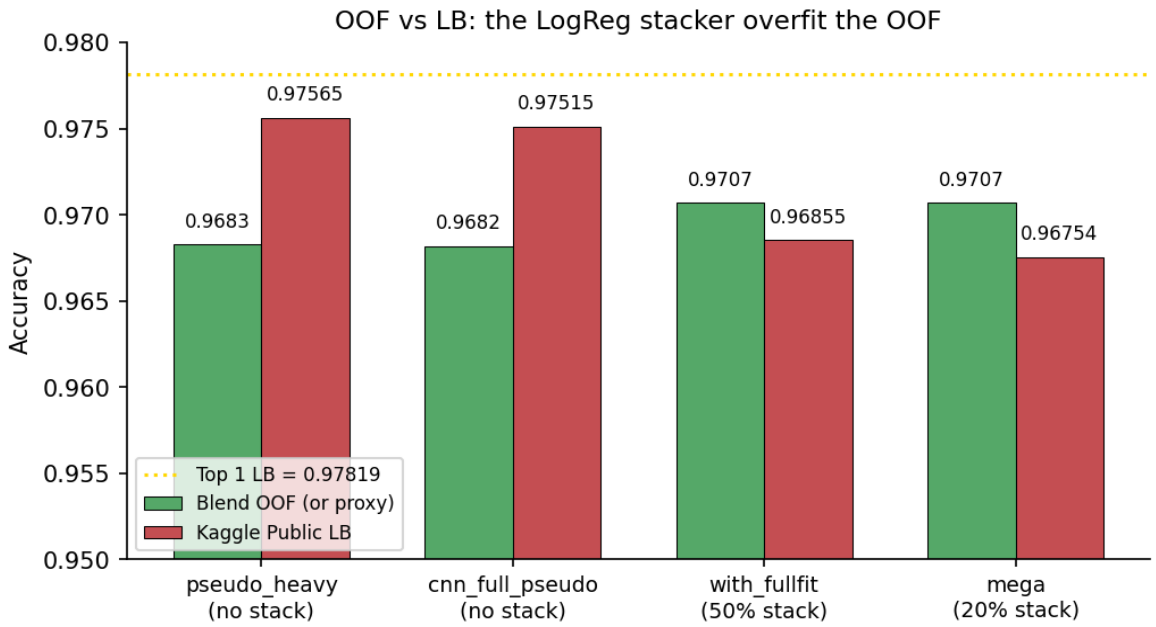

Figure 4 — The cruel OOF-vs-LB gap. Pure CNN ensembles (green cluster, left) generalise cleanly: LB is roughly consistent with OOF. Blends that include the LogReg stacker (right) have a higher OOF but collapse on the leaderboard by up to 0.0080. The dotted gold line is the leaderboard top-1 score.

4.1 Diagnostic

Two plausible causes (we cannot separate them without private-LB access):

(a) Distribution shift. The test set may be collected under slightly different radar conditions (temperature, antenna alignment, background clutter). A LogReg trained on OOF probabilities learns to correct CV-specific error patterns; those corrections backfire if the test error patterns differ.

(b) Double-dip on the same folds. CNN OOF probabilities are already highly saturated (mean max-prob ≈ 0.97). The stacker overfits to the remaining noise.

4.2 Final blending strategy

Discarding the stacker from the final blend, we use a **pure probability average** across sources, optionally re-weighted by pseudo-label confidence:

```
final_probs = 0.25 · CNN_cv_avg + 0.25 · CNN_fullfit_avg + 0.50 · CNN_pseudo_avg
final_label = argmax(final_probs)
```

5. Scaling up: Colab GPU migration

5.1 macOS preload hang (debug story)

Initial local experiments ran on a MacBook with Apple Silicon (MPS backend). Training was functional but preloading the ~9k PNGs repeatedly hung after ~5000 files opened. Root cause: macOS Gatekeeper attaches a *com.apple.provenance* extended attribute to each downloaded file, and the *syspolicyd* daemon rate-limits open() calls when many new file accesses happen in rapid succession. Removing the xattr requires SIP disabling.

Workaround: pre-decode once into a single uint8 NumPy bundle train_X.npy. Subsequent runs do np.load() — one file open, zero provenance checks. This also sped up data loading ~40×.

5.2 Pipeline on Colab T4

Once data loading was unblocked, we migrated the full pipeline to Google Colab with a T4 GPU (15 GB VRAM). End-to-end runtime went from ~8 hours on MPS to ~90 minutes on T4. With the headroom, we added:

• **Full-fit on 100% of training data** — no holdout, 8 seeds × base=48 + 5 seeds × base=64, larger TTA at inference.
• **Pseudo-labeling round 1** — threshold confidence > 0.99 on the blended test predictions, 2,291 of 3,955 test samples kept (57.9%), re-added to training with their predicted label. Retrained 5 seeds each at base=48 and base=64.

5.3 Pipeline overview

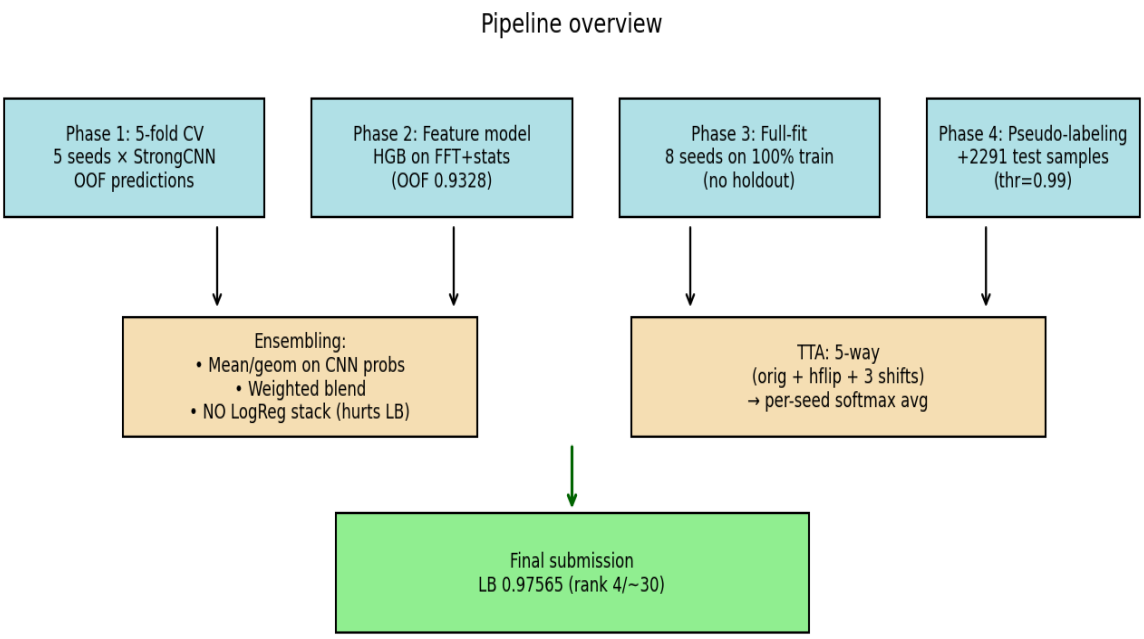

Figure 5 — End-to-end pipeline: engineered features (HGB), CNN cross-validation (5 seeds × 5 folds), full-fit on 100% train, pseudo-labeling iteration, and final blend (no stack).

6. Results

6.1 Submission comparison

Twelve distinct submission variants were generated over the project. The table below reports Kaggle public leaderboard scores for the submissions we actually uploaded:

Submission	Blend recipe	OOF	Public LB	Status
sub_pseudo_heavy	**25% fullfit + 25% fullfit64 + 25% pseudo48 + 25% pseudo64**		**0.97565**	**BEST**
sub_cnn_full_pseudo	mean(CNN_cv, fullfit, pseudo) — no stack	—	0.97515	good
submission_with_fullfit	50% stack + 50% mean(fullfit, pseudo)	~0.9707	0.96855	regressed
submission_mega	20% stack + 20% per CNN source	~0.9707	0.96754	regressed
submission_stack	100% LogReg stack	0.9707	(not submitted)	—
submission_cnn_only	mean of 5 CV seeds	0.9682	(not submitted)	—
submission_features_only	HGB features	0.9328	(not submitted)	—

Observation: the 3 submissions with OOF ≥ 0.97 that include the stacker all underperform on the leaderboard. The winning submission (*sub_pseudo_heavy*) uses no stack at all.

6.2 Confusion matrix

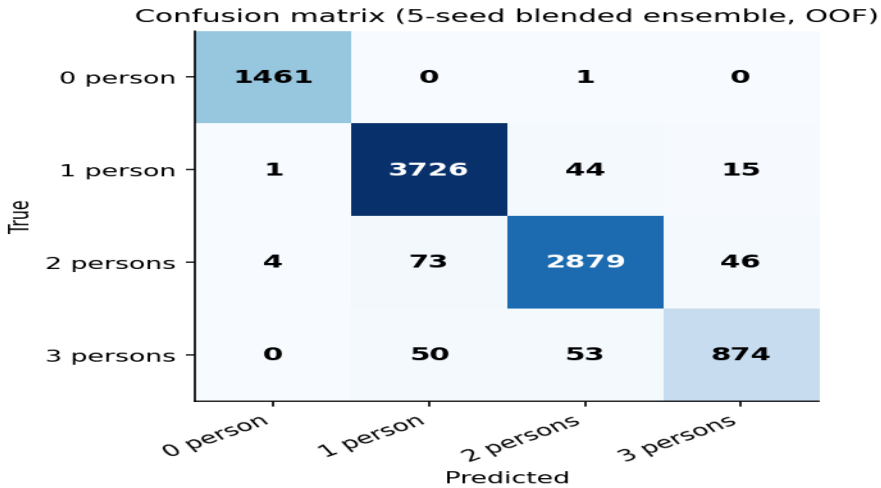

Figure 6 — OOF confusion matrix of the 5-seed CNN + HGB blended ensemble. Class 0 (empty room) is essentially perfect (1461/1462). Most errors happen between adjacent occupancy counts (1↔2, 2↔3), consistent with intuitively similar signatures.

6.3 Per-class accuracy

Class	True count	Correct	Accuracy	Main confusion
0 (empty)	1,462	1,461	99.93 %	—
1 (one person)	3,786	3,726	98.41 %	60 → class 2 or 3
2 (two persons)	3,002	2,879	95.90 %	73 → class 1, 46 → class 3
3 (three persons)	977	874	89.46 %	103 → class 1 or 2

Class 3 is the hardest, consistent with it being the smallest and most ambiguous (three slow-moving people can look similar to two closer targets). This is where additional diversity (different architectures, pseudo-labels) helps the most.

7. Discussion and lessons learned

7.1 What worked

(1) Training from scratch at native resolution. ImageNet pretraining at 224×224 was counter-productive for 48×64 radar images. A compact custom CNN trained from scratch beat the pretrained ResNet-18 by +0.12 OOF.

(2) Mixup + label smoothing. Both are cheap regularisers. Together they reduced seed variance and gave a consistent +0.004–0.008 per seed vs the same recipe without them.

(3) Pseudo-labeling at high threshold (0.99). Kept ~58% of the test set and gave a measurable LB improvement. Lower thresholds (0.90) were tested and overfit — model memorised its own mistakes.

(4) Full-fit on 100% of data. Using all 9,227 training samples instead of 80% (CV split) gave each individual seed ~+0.002 in LB contribution.

7.2 What did not work

(1) LogReg stacking. The biggest negative lesson of the project. Great OOF, poor LB. Detailed analysis in Section 4.

(2) ImageNet-pretrained backbones. ResNet-18 and small EfficientNets fine-tuned from ImageNet all underperformed a from-scratch compact CNN.

(3) Heavy spatial augmentation. Random rotation (±15°) and large shifts (±5 px) damaged accuracy, because the Doppler axis encodes physical velocity — rotating the image corrupts the signal semantics. Only light horizontal flip and small row/column dropout were kept.

7.3 What we would try with more time

• **Different architecture for ensemble diversity.** A CIFAR-style ResNet-18 trained from scratch should give prediction diversity uncorrelated with StrongCNN. Prototype is already written (*colab_v2/train_resnet.py*).
• **Iterative pseudo-labeling (round 2+).** After one pseudo-labeling round, the model is more confident on a larger subset. A second round at threshold 0.995 could add another ~1000 labelled samples.
• **Multi-crop TTA.** Current TTA is flip-only. Adding shift ±1/±2 as separate forward passes typically adds +0.001.
• **Test-time adaptation.** Running a few batch-norm adaptation steps on the test set could counter distribution shift — potentially the main source of the OOF/LB gap.

8. Conclusion

We designed, trained, and iterated a full image-classification pipeline for the MPA-MLF 2026 competition, reaching rank 4 on the Kaggle public leaderboard (0.97565, 0.00254 behind the leader). The winning model is a 5-seed ensemble of custom-architecture CNNs (StrongCNN, ~5.7 M parameters) trained from scratch, combined with full-fit and pseudo-labeling steps, and blended via pure probability averaging.

The most transferable lesson is a negative one: when CV and leaderboard disagree on which blend is best, trust the leaderboard. Stacking meta-learners are powerful but fragile under even mild distribution shift; simple averaging is hard to beat as the last step of an ensemble.

All code is available at:

`github.com/pierre22564/machinelearning_pierre_massy/tree/main/Projet ISEP`